

An introduction to Python language

Constructor

Dr. Ahmed H. Aliwy

University of Kufa

2020

Global and local variables

Global variables

In Python, a variable declared outside of the function or in global scope is known as global variable. This means, global variable can be accessed inside or outside of the function.

```
def f():  
    print(x)  
x=10  
f()
```

↓
Output

10

we created **x** as a **global variable** and defined a **f ()** to print the global variable **x**.

Global and local variables

Global variables

All the defined variables inside function are local

```
def f():  
    x=20  
    print(x)  
x=10  
f()  
print(x)
```

Output

```
20  
10
```

Global and local variables

Global variables

Using local and global variable cause problem to compiler

```
def f():  
    print(x)  
    x=20  
    print(x)  
x=10  
f()  
print(x)
```

Output

```
UnboundLocalError: local variable 'x' referenced  
before assignment
```

Global and local variables

Global variables

```
x = 2
def f():
    x = x * 2
    print(x)

f()
```

← Errors

Solution →

```
x = 2
def f():
    global x
    x = x * 2
    print(x)

f()
```

Output
4

Passing a function as an argument to another function

❑ Functions can be referenced

```
>>> def power(x,y):  
    print(pow(x,y))  
  
>>> power  
<function power at 0x02310468>
```

❑ Since we can refer to functions like any other object, we can point a variable to a function

```
def square(n): return n**2  
def cube(n): return n**3  
operations = [square, cube]  
for action in operations:  
    print(action(3))
```

Output



```
9  
27
```

Passing a function as an argument to another function

Function calls tree functions where their names as argument

```
def power(x,y):  
    print(pow(x,y))  
def mod(x,y):  
    print(x%y)  
def add(x,y):  
    print(x+y)  
def operation(f,x,y):  
    return (f(x,y))  
operation(power,10,2)  
operation(mod,10,3)  
operation(add,10,2)
```



Output

```
100  
1  
12
```

Lambda expression

Function calls tree functions where their names as argument

lambda arguments: expression

- * Lambda forms can take any number of arguments but return just one value in the form of an expression. They cannot contain commands or multiple expressions.
- * An anonymous function cannot be a direct call to print because lambda requires an expression.
- * Lambda functions have their own local namespace and cannot access variables other than those in their parameter list and those in the global namespace.

Lambda expression

Comparing of using lambda and normal function for returning the square of a number.

```
def sequare(x):  
    return(x*x)  
g=lambda x:x*x  
print(g(3))  
print(sequare(3))
```

Output

```
9  
9
```

Passing a function as an argument to another function

Extra examples on lambda expressions

```
f=lambda x,y:x+y
g=lambda x,y=5,z=2: x*y*z
h=lambda *x:x
n=lambda x,y: ((x,y),x**2,y**2)
print(f(2,3))
print(g(2))
print(g(2,3,3))
print(h(2))
print(h(1,2,"Ali"))
print(n(2,3))
m,x,y=n(2,3)
print(m,x,y)
```



Output

```
5
20
18
(2,)
(1, 2, 'Ali')
((2, 3), 4, 9)
(2, 3) 4 9
```

- ☐ f take two arguments and return the sum
- ☐ g take three arguments (two of them are defaults) and return the product
- ☐ h take variables number of arguments and return of them
- ☐ n take two arguments and return three results (these arguments, square of the first, square of the second)